

C-programmering

"The first and most important thing of all, at least for writers today, is to strip language clean, to lay it bare down to the bone."

-Ernest Hemingway

1

C-programmering

Programspråket C

- Skapades ~1972 av Dennis M. Ritchie
- Implementerades ursprungligen på en PDP-11
- Stora framgångar de senaste 10 åren. Mycket tack vare UNIX
- Började standardiseras 1982 ANSI-C, (X3J11)
- Utvecklas mot C++
- Designades som ett maskinnära språk vars typer direkt överensstämmer med hårdvaran

2

C-programmering



3

C-programmering

Till ett husbygge behövs

- byggmaterial
- ritningar

Till ett program behövs

- data (variabler)
- instruktioner (kod eller funktioner)

4

/ Detta är ett C-program */*

```
#include <xxx> //Preprocessor-
#define zero 0 //kommandon
int i; //Definition och dekl
float a,b,c;
void main (void) // Funktion
{
    while (1) // Satser
    {}
    a=b*c; // Uttryck
}
```

5

Preprocessorn

- Preprocessorn är ett rent text till text verktyg.
- Några möjliga direktiv:

```
#include <filnamn>
#define <name> <definition>
#if - else - endif
```

6

Preprocessorn

```
#include <filnamn>
```

Två varianter:

```
#include <stdio.h> - inkluderar en  
fil från systemkatalogen  
#include "fil1.h" - inkluderar en  
fil från arbetskatalogen
```

7

Preprocessorn

```
#define <name> <definition>
```

Konstanter

```
#define MAXINT 32767  
#define MITTNAMN "Avo Kask"
```

Makro funktioner

```
#define max(a,b) ((a) > (b) ? (a) : (b))  
#define dprintf skriv_pa_display
```

8

Preprocessorn

Vad som helst (krafs)

```
#define BEGIN {  
#define END   }  
#define om    if  
#define annars else
```

9

Definitioner och deklarationer av objekt i C

- Alla objekt måste definieras
- Att definiera betyder att göra en "länk" mellan objektets identifierare och en lagringsklass samt en typ
- Deklaration menas att "göra synlig"

10

Definitioner och deklarationer av objekt i C

```
int i;  
float pi = 3.1415;  
unsigned char alfa, beta;  
double resultat;  
float temperatur();  
void display();  
....
```

11

Numeriska konstanter

Kan skrivas i tre talsystem:

```
Hexadecimalt : 0x3FFF (Observera `0x´)  
Decimalt      Som vanligt 1023  
(Oktalt)
```

12

Objekt och identifierare

- Objekt upptar minne
- Minsta objektet är en byte
- Namnet på ett objekt = identifierare
- Identifierare skrivs med bokstäver, siffror och _
- I identifierarnamn görs skillnad mellan gemener och versaler
- Identifierare får inte :
 - Börja med en siffra
 - Inte använda reserverade ord eller symboler

13

Datatyper

- Det finns fem grundläggande datatyper
 1. **char** t.ex ASCII-tecken (1 byte)
 2. **int** default typ (2 byte)
 3. **float** ex. 12,5 (4 byte)
>1100.1=0.11001*2¹¹⁰
 4. ***pekare** pekare
 5. **void** 'ingenting'

14

Datatyper

- En datatyp kännetecknas av att den :
 - Upptar minnesutrymme
 - Kan vara olika för olika kompilatorer
 - **Char** och **int** KAN DEFINIERAS MED TECKEN **signed** ELLER UTAN TECKEN **unsigned**.
 - **Char** och **int** kan också definieras som **long** eller **short**.
 - **Float** kan också finnas med dubbel precision - **double** (8 bytes)

15

Datatyper

Fördefinierade
char, **int**, **float**, **double**

Prefix till heltal
short, **long**, **signed**, **unsigned**

Exempel:
short int, **long int**, **unsigned int**

16

Datatyper

Char	-128	127
Int	-32 768	32 767
Short int	-32 768	32 767
Long int	-2 147 483 648	2 147 483 647
Unsigned char	0	255
Unsigned int	0	65 535
Unsigned short int	0	65 535
Unsigned long int	0	4 294 967 295
Enum	0	65 535
Float	3,4E+38	3,4E-38
double	1,7e+308	1,7e -308
Long double	1,7e+308	1,7e -308

17

Datatyper

Uppräkningsbara typer
enum {svart, vit, gul} farg;
Fält **char** vektor[100];

Notera: Första indexet i ett fält är alltid 0.

18

Datatyper

Poster (struct)

```
struct {  
    int antal;  
    char tillverkare[10];  
    float pris;  
} resistorer; // variabeln
```

19

Datatyper

Bitfält i strukturer används för att:
Packa heltalskomponenter
Namnge specifika bitar i ett ord

Exempel:

```
struct status  
{  
    unsigned a:2;  
    unsigned b:6;  
    unsigned c:8;  
} kontroll;
```

20

Datatyper

Datatyper "BOOLEAN" finns ej

Istället används heltal med konventionen:

```
0 == FALSE  
!0 == TRUE
```

21

Textsträngar

Inga speciella strängtyper
Inga inbyggda operationer
Refereras normalt med hjälp av pekare
Avslutas med '\0'

22

Uttryck - operander och operatorer

```
...  
ADCON = 0x0090;  
    /* starta A/D kanal 0 */  
while (ADCON & 0x0100==0x0100);  
    /*vänta till A/D klar */  
temp =ADDAT;  
    /* lagra temporärt*/  
return (5.0*(temp&0x03FF)/1024.0);  
    /* returnera A/D i mV*/  
....
```

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADSCF	ADIFSCF	ADIF	ADIF	ADIFSC	ADIFSCF	ADIF	ADIF	ADIFSC	ADIFSCF	ADIF	ADIF	ADIFSC	ADIFSCF	ADIF	ADIF

23

Uttryck - operander och operatorer

- Uttryck kan bestå av:
 - En operand och en operator eller
 - Flera operander och operatorer
- Operanderna kan vara av olika typ
- *Var uppmärksam på detta*

24

Operander och operatorer

- DE 'VANLIGASTE' OPERATORERNA ÄR:

- tilldelning	=	+=	-=
- logiska, bitvis shift	>>	<<	
• och	&	bitvis AND	
• eller		bitvis OR	
• negation	~	bitvis komplement	
• exklusivt eller	^	bitvis exor	
- logiska I/O	&& AND OR ! NOT		
- relationer	< > >= <= == !=		
- inkrement/dekrement	++ --		
- aritmetiska	* / + - %		

25

Operander och operatorer

Kan användas på int och char

Exempel:

```
int a,b;
a = 1 << 5;
b = a ^ b;
a |= b;
```

26

Operander och operatorer

- Uttrycken 'löses' upp normalt i riktning höger till vänster (eller tvärtom beroende på operatorm)
- Operatörerna har olika prioritet
- Det finns 15 olika prioritetsregler i C.
- Dessa kan "ersättas" med följande 2 :

* och / kommer före + och -
Sätt parenteser runt allt annat

27

Typomvandlingar

- Operanderna måste vara av samma typ före operationen
- Man skiljer på automatiska och explicita typomvandlingar
- Explicit typomvandling (eng. casting)
(typnamn) operand * operand
 - den vänstra operanden kommer att omvandlas till den typ som beskrivs inom parentesen
- Ex : **(int)a * b**

28

Reserverade symboler och ord

! connective NOT	logiskt ICKE
# preprocessor sign	tecken för preprocessor-kommandon
% modulus operator	rest vid heltalsdivision
^ bitwise XOR	exklusivt OR
& bitwise AND	logiskt AND bit för bit
& addressoperator	adressoperator
&& connective AND	AND - sant eller falskt
* multiplikation	multiplikation
* pointer	pekare

29

Reserverade symboler och ord

()	parentese	parentes
-	subtraction	subtraktion
--	decrement	minska med ett
->	structure pointer	pekare i poster
+	addition	addition
++	increment	öka med ett
=	assign	tilldelning
==	equal to	lika med (jämförelse)
!=	not equal to	inte lika med (jämförelse)

30

Reserverade symboler och ord

~	bitwise not	ICKE bit för bit
{ }	block parenthese	block parentes
[]	array paranthese	fält parentes
\	escape character	escape tecken
	bitwise OR	ELLER bit för bit
	connective OR	ELLER sant eller falskt
;	statement delimiter	avslutning av sats
:	delimiter i case statement	avslutning i case-sats
"	double quotes for string constants	
'	ASCII-character	

31

Reserverade symboler och ord

<	less than	mindre än
<<	left bitwise shift	vänster shift bitvis
>	greater than	större än
>>	right shift	höger bitvis shift
,	delimiter between objects	skiljetecken mellan objekt
/	division	division
/*	comments	skiljetecken för kommentarer
//	comments	skiljetecken för kommentarer
?	conditional expression	villkorsoperator

32

Reserverade symboler och ord

• Följande ord är reserverade för C-syntax:
auto, break, case, char, const, continue, default, do, double, else, enum, extern, float, for, goto, if, int, long, register, return, short, signed, sizeof, static, struct, switch, typedef, union, unsigned, void, volatile, while.

**KAN DU BETYDELSEN AV DESSA
 SYMBOLER OCH ORD KAN DU DET
 MESTA AV SYNTAXEN I C!**

33

De sex grundläggande satserna

1. UTTRYCK
2. SAMMANSATTA SATSER eller BLOCK SATSER { }
 allt som finns inom { } betraktas som en sats avslutas **ej** med ;
3. VILLKORSSATSEN - if
4. FLERVALSSATSEN - switch - case
5. REPETITION - while
6. REPETITION - for

34

De sex grundläggande satserna

1. UTTRYCK

```
a = b * c;
temp = ADDAT;
```

35

De sex grundläggande satserna

2. SAMMANSATTA SATSER eller BLOCK SATSER { }

```
void main(void)
{
    while (1)
    {
        a++;
    }
}
```

36

De sex grundläggande satserna

3. VILLKORSSATSEN - if

```
if ( uttryck)      if (ADDAT==1023)
  sats;            temp++;
else              //else kan uteslutas
  sats;
```

uttryck skall vara en skalär
där sant skilt från 0 och falskt = 0

37

De sex grundläggande satserna

4. FLERVALSSATSEN - switch

```
switch (uttryck)
{
case 'uttryck': sats; break;
case 'uttryck': sats; break;
case 'uttryck': sats; break;
default: sats; break;
}
```

38

De sex grundläggande satserna

4. FLERVALSSATSEN – switch - Exempel

```
switch (HASTIGHET)
{
case 0: PW0=50; break;
case 1: PW0=100; break;
case 2: PW0=255; break;
default: PW0=0; break;
}
```

39

De sex grundläggande satserna

5. REPETITION - while

```
while ( uttryck)    while(1) {
  sats;              ++x;}
```

uttryck skall vara en skalär
där sant skilt från 0 och falskt = 0

40

De sex grundläggande satserna

6. REPETITION - for

```
for ( uttryck1; uttryck2; uttryck3)
  sats;
```

uttryck1 är startvillkor för räknaren
uttryck2 är tillsvidarevillkor
uttryck3 är upp/ned räkning av räknaren

41

De sex grundläggande satserna

6. REPETITION – for Exempel

```
for ( a=1; a<10000; a++)
  P3_0=0;
for ( a=1; a<10000; a++)
{ P3_0=1;
  P3_1=~P3_1;
}
```

42

Funktioner

- EN FUNKTION KAN BETRAKTAS SOM EN SUBROUTIN
- Funktionsdefinition
 - måste ligga yttersta lagret av programmet
 - ej inne i andra funktioner
 - bara en definition per program
 - definitionen innebär att minne reserveras

43

Funktioner

– Exempel :

```
int adam(char a, char b); //funktionsdekl.
```

- typ på returnerat värde är **int**
- funktionens identifierare **adam**
- **a** och **b** är formella parametrar
- Funktionen måste vara deklarerad innan den anropas
- Om anrop från andra källkodsfiler används prefixet
 - **extern**

44

Funktioner

– Exempel :

```
//funktionsdefinition
int adam(char a, char b)    //funktionshuvud
int y;
{                            //funktions kropp
    y=a*b;
}
```

45

Funktioner

– Exempel :

```
//funktionsanrop
char x, char z;
int y;

y = adam(x, z);
```

46

Jämförelse mellan C och assembler

```
/* C program för 167 - komplementera */
P2=~P2 ;
MOV    R12,P2      // 2 bytes
CPL    R12         // 2 bytes
MOV    P2,R12      // 2 bytes
; assembler
CPL    P2          // 2 bytes
```

- assemblatorn producerar 2 byte av denna instruktion.
- kompilator översätter detta med 3 instruktioner om tillsammans 6 byte

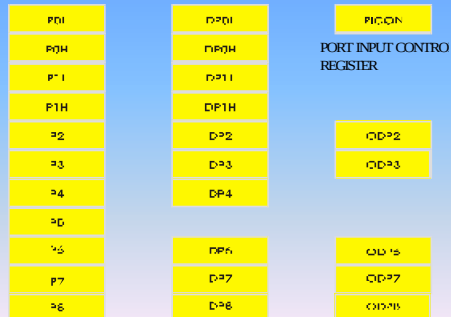
47

Exempel på C-program

```
//main.c, programkod för att tända LED P3.0 med
//switch på port 5.6
```

48

PORTAR-register



49

Exempel på C-program

```
//main.c, programkod för att tända LED P3.0 med
//switch på port 5.6
#include <mm167reg.h>

void main(void)
{
    ODP3 |= 0x0000; // Port 3 är satt till
                    //push/pull, 1 ger open drain
    DP3 |= 0x01;    // Bit 0 är satt till utgång
}
```

50

Exempel på C-program

```
while(1) //Loopa forever
{
    P3_0=P5_6; //Värdet i P5_6 överförs till P3_0
}
}
```

51

EX : AD-omvandling av potentiometer signal

52

AD-register



P5 = AD PORT IN
 ADDAT = AD RESULTAT
 ADDAT2 = AD-RESULTAT CHANNEL INJECTION
 ADCON = AD KONTROLLREGISTER
 ADCIC = AD INTERRUPT KONTROLL REGISTER
 ADCIC = AD INTERRUPT KONTROLL REGISTER

53

AD-omvandling av potentiometer signal

```
#include <mm167reg.h>

void main(void)
{
    DP3 |= 0x03FF; // OBS !!! OR-tecknet här
                  // konfigurerar port 3 som utport där
                  // en diodarray är ansluten
    ADCON = 0x0090; // konfigurerar AD-omvandlaren
                  // Ch0, go, cont
    while(1) //evighetsslinga
    {
        P3 = ADDAT; //sätter port tre till ADvärdet
    }
}
```

54

Styrning av motor med PWM

55

PWM-register



56

PWM-register

PWMCON0 (PF20 ₀ / 0B ₀)															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PIR0	PIR1	PIR2	PIR3	PIR4	PIR5	PIR6	PIR7	PIR8	PIR9	PIR10	PIR11	PIR12	PIR13	PIR14	PIR15
0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000

Bit	Function
PTRx	PWM Timer x Run Control Bit 0: Timer PTx is disconnected from its input clock. 1: Timer PTx is running
PTIx	PWM Timer x Input Clock Selection 0: Timer PTx clocked with CLP _{CKU} 1: Timer PTx clocked with CLP _{CKU} / 64
PIEx	PWM Channel x Interrupt Enable Flag 0: Interrupt from channel x disabled 1: Interrupt from channel x enabled
PIRx	PWM Channel x Interrupt Request Flag 0: No interrupt request from channel x 1: Channel x interrupt pending (must be reset via software)

57

PWM-register

PWMCON1 (PF20 ₁ / 0B ₁)															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PIR16	PIR17	PIR18	PIR19	PIR20	PIR21	PIR22	PIR23	PIR24	PIR25	PIR26	PIR27	PIR28	PIR29	PIR30	PIR31
0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000

Bit	Function
PIRx	PWM Channel x Output Enable Bit 0: Channel x output signal disabled, generates interrupt only 1: Channel x output signal enabled
PIRx	PWM Channel x Mode Control Bit 0: Channel x operates in mode 0, i.e. edge aligned PWM 1: Channel x operates in mode 1, i.e. center aligned PWM
PIR1	PWM Channel 0/1 Burst Mode Control Bit 0: Channels 0 and 1 work independently in respective standard mode 1: Outputs of channels 0 and 1 are ANDed to POUT0 in burst mode
PIRx	PWM Channel x Single Shot Mode Control Bit 0: Channel x works in respective standard mode 1: Channel x operates in single shot mode

58

PWM-register

```
// Motorn styrs med 20 ms pulser
#include <mm167reg.h>

void main(void)
{
    DP7_0 = 1;
    P7_0 = 0;
    // konfigurerar port 7_0 som utport där motorn
    // är ansluten
    PWMCON1 = 0; //alla kanaler off
    //PP0 = PERIOD skall bestämmas
    //PW0 = DUTYCYCLE skall bestämmas
```

59

PWM-register

PWMCON0 (PF20 ₀ / 0B ₀)															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PIR0	PIR1	PIR2	PIR3	PIR4	PIR5	PIR6	PIR7	PIR8	PIR9	PIR10	PIR11	PIR12	PIR13	PIR14	PIR15
0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000

PWMCON1 (PF20 ₁ / 0B ₁)															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PIR16	PIR17	PIR18	PIR19	PIR20	PIR21	PIR22	PIR23	PIR24	PIR25	PIR26	PIR27	PIR28	PIR29	PIR30	PIR31
0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000

```
// PTR = start/stop timer (1/0)
// PTI0 = clock in fcpu/(fcpu/64) (0/1)
// PIE0 = interrupt enable/disable (1/0)
// PIR0 = interrupt request no/yes (0/1)
PWMCON0 = 11;
PWMCON1 = 1; // enable P7_0
PP0=6254;
// PERIOD : 64*50*X = 20 000 000 ( 20 ms )
PW0=PP/2; // DUTYCYCLE = PERIOD/2
while(1){
}
```

60

Finessen med C : Pekare

- En pekare är ett objekt och måste således definieras
- Alla objekt har en adress i minnet. En pekare gör det möjligt att hitta objektet indirekt.

61

Finessen med C : Pekare

Att definiera en pekare:

```
<typ> * <variabel>   int *x_pek;
```

Åtkomst av ett element:

```
* <variabel>          *x_pek = 2;
```

Refererande av variabler

```
& <variabler>        x_pek = &x;
```

62

Pekare

Exempel:

```
int x;  
int *x_pek;  
x = 1;  
x_pek = &x;  
*x_pek = 2;  
/* x == 2 */
```

63

Pekare

```
int x;           // variabel  
int *x_pek;      // pekarvariabel
```

* säger oss att **x_pek** är en pekare

x_pek är pekarvariabeln

int specificerar **typen** som pekaren pekar på, dvs 16-bitar

64

Pekare

Operator * : givet en pekare, hämta innehållet

Operator & : givet en variabel, peka på den (adressen till)

```
int *x_pek; // def av pekarvariabeln  
x_pek = &a; //adressen till a lagras i px  
b=*x_pek;  // b sätts lika med a
```

65

Fält och strängar

- Fält och sträng är ett sätt att lagra objekt av samma typ och med samma 'namn'.
- Fält och strängar är också objekt och måste definieras
- Ett fält och sträng indexeras med första elementet som noll.

66

Fält och strängar

```
char buffer [80];
char tabell [2] = {10,5};
• initiering av fält sker på detta sätt
tabell [0] är lika med 10
tabell [1] är lika med 5

int matris[2][4]= { 1, 2, 3, 4},
                  {10,20,30,40}
                  };
```

67

Fält och strängar

```
buffer [0]= 'a';
• buffer är egentligen en pekarkonstant till första elementet i fältet
• pek är en pekarvariabel som kan användas för att nå valfritt element i buffer
pek == buffer
pek == &buffer[0]
*pek == buffer [0]
char *pek;
pek = buffer;
```

68

Fält och strängar

```
int a[100];
int *pa;
//Låt pa peka a:s i:te element
pa = &a[i];
//Första elementet
pa = &a[0];
//Finess...
pa = a;
//Dvs, a är dels namnet på en vektor med 100 element, dels en pekare till element 0
```

69

Fält och strängar

Ekvivalenta uttryck

```
int a[100], i
a == &a[0]
a+i == &a[i]
*(a+i) == *&a[i] == a[i]
*(a+i) == *(i+a) == i[a]
```

70

Fält och strängar

Pekar-aritmetik:

```
int a[10], *pek;
pek = &a[3];
pek = pek + 1;
//eller pek++ eller pek += 1;
Vad pekar pek på? a[4];
dvs pek = &a[4];
Analogt med pek--;
```

71

Variabla typer - Poster (struct)

Poster används när man vill hantera objekt av olika typ

```
struct
{
    char reg_nr [6], farg [10];
    int arsmoell , vikt ;
} bil; // variabeln
```

72

Variabla typer - Poster (struct)

För att komma åt ingående delarna används punktnotation

```
bil.vikt = 1050;
```

Postvariabelnamnen är 'riktiga variabelnamn' (inte bara referens till första objektet)

73

Variabla typer - Poster (struct)

Pekare till poster, i detta fall bil

```
struct bil *bilp;  
(*bilp).vikt = 1050;
```

Samma som

```
bil.vikt = 1050;
```

74

Variabla typer - union

- Minne reserveras bara för den största ingående typen
- Variabeln kan bara innehålla en av de definierade objekten
- Selektion med punktnotation
- oj.I alt. Oj.F alt. Oj.C[0]

75

Variabla typer - union

```
union exempel  
{  
    int i;  
    float f;  
    char c[10];  
} oj;
```

variabeln oj kommer endast att innehålla en av i, f eller c

76

Egna typer - typedef

Egna typer kan definieras med **typedef**

```
typedef char tecken;  
typedef int flt [20], *fltp;  
  
tecken v1,v2,v3;  
flt f1,f2,f3;  
fltp p1;
```

77

Egna typer – enum

```
typedef enum tecken {red,green};  
switch (status)  
{  
    case 'red': a=b;break;  
    case 'green':a=c;break;  
}
```

Enum variabeln kan användas som en int

78

Egna typer – enum

```
include <fil.h>
enum thevenin {source=12,resistans=150}

void main(void)
float strom,last=100;
{
    strom=source/(resistans+last);
}
```

79